2

SESSION 2

Functions, Collections & Error Handling

90 minutes

- Quick recap of Session 1
- Function parameters — positional, named & optional
- Arrow functions and functions as first-class objects
- Closures, scope & recursion

31

SESSION 2 AGENDA

What We'll Cover Today

1. Quick recap of Session 1
2. Function parameters — positional, named & optional
3. Arrow functions and functions as first-class objects
4. Closures, scope & recursion
5. List, Map & Set in depth
6. Generics — writing reusable, type-safe code
7. Handling errors: try/catch/finally & custom exceptions
8. A first look at Future & async/await

32

RECAP

Quick Recap: Session 1

- Dart is statically typed with sound null safety (String vs String?)
- var / final / const each lock in a value differently
- if/else and switch for branching, plus the ternary shortcut
- for, for-in, forEach, while, do-while for repetition

33

2.4.1 ANATOMY

Functions Deep Dive

```dart
String makeGreeting(String name) {
  return 'Hello, $name';
}
```

- Pattern: ReturnType functionName(ArgType arg) { body }
- Functions must return a value that matches the declared type
- Every function is an object of type Function in Dart

34

2.4.2 PARAMETERS

Positional & Named Parameters

```dart
void log(String msg, int line) { ...
}
log('A bug!', 55);
// positional — order matters
```

```dart
void log({String msg, int line}) {
... }
log(msg: 'A bug!', line: 44);
// named — any order
```

- Positional args must be supplied in the exact declared order
- Named args ({ }) are matched by label — order doesn't matter
- @required (or required in Dart 2.12+) makes a named param mandatory

2.4.2–2.4.3 PARAMETERS

Optional Positional & Defaults

```dart
int addNums(int x, int y, [int z = 0]) {
  return x + y + z;
}
addNums(5, 4);      // 9   (z defaults to 0)
addNums(5, 4, 3);   // 12
```

- Square brackets [ ] mark optional positional parameters
- = value supplies a default if the caller omits the argument
- Named and default values can combine: {int z = 0}

ARROW FUNCTIONS

Arrow & Anonymous Functions

```
String greet(String name) => 'Hi, $name';

var square = (int n) => n * n;   // anonymous fn
print(square(5));              // 25
```

- => expr; is shorthand for { return expr; }
- Only valid for a function body that's a single expression
- Anonymous functions have no name — assign them to a variable or pass inline

37

HIGHER-ORDER FNS

Functions Are Objects

```
void runTwice(Function fn) {
  fn();
  fn();
}
runTwice(() => print('Hi!'));  // Hi! Hi!
```

- A higher-order function takes a function as an argument, or returns one
- list.forEach(), .map(), .where() are all higher-order
- This is how callbacks (onPressed, onTap...) work in Flutter

38

SCOPE

Closures: Functions That Remember

```
Function makeCounter() {
  int count = 0;
  return () => ++count;   // captures count
}
var counter = makeCounter();
counter();  // 1   counter();  // 2
```

- A closure is a function bundled with its surrounding scope
- Each call to makeCounter() creates a fresh, independent count
- Dart is lexically scoped: inner blocks read outer variables

39

PRACTICE

Recursion in Dart

```
int factorial(int n) =>
    n <= 1 ? 1 : n * factorial(n - 1);

print(factorial(5));  // 120
```

- A recursive function calls itself with a smaller sub-problem
- Always needs a base case, or it recurses forever (stack overflow)
- Elegant for tree-shaped data — exactly how Flutter's widget tree renders

40

COLLECTIONS

List in Depth

```
var nums = [3, 1, 4, 1, 5];
nums.add(9);            // [3,1,4,1,5,9]
nums.remove(1);         // removes first 1 found
nums.sort();            // ascending, in place
nums.contains(4);       // true
```

- Lists are ordered and index-accessible: nums[0]
- add/remove/sort mutate the list in place
- List.filled, List.generate build lists programmatically

41

COLLECTIONS

List in Depth

Side-by-side comparison

	`List.filled`	`List.generate`
Value per slot	Same value, repeated	Computed per index
Growable default	`false`	`true`
Good for	A fixed placeholder (zeros, blanks)	Indexed, computed, or per-slot-unique data

42

FUNCTIONAL STYLE

Powerful List Methods: map, where, reduce

```
var nums = [1, 2, 3, 4, 5];
nums.where((n) => n.isEven);      // (2, 4)
nums.map((n) => n * 2);           // (2,4,6,8,10)
nums.reduce((a, b) => a + b);     // 15
nums.fold(100, (acc, n) => acc + n); // 115
```

- where filters, map transforms — both return a lazy Iterable
- reduce combines elements using the list's own first value
- fold works like reduce, but you supply the starting value

43

COLLECTIONS

Map in Depth

```
var ages = {'Nora': 30, 'Sam': 25};
ages['Nora'];              // 30
ages['Alex'] = 40;      // adds a new entry
ages.containsKey('Sam'); // true
ages.forEach((k, v) => print('$k: $v'));
```

- Map<K, V> stores key → value pairs; keys are unique
- Square-bracket access returns null for a missing key
- Perfect fit for JSON data once we hit dart:convert

44

COLLECTIONS

Set in Depth

```
var a = {1, 2, 3}, b = {2, 3, 4};
a.union(b);          // {1, 2, 3, 4}
a.intersection(b);   // {2, 3}
a.difference(b);     // {1}
{1, 1, 2, 2, 3}.length;  // 3 — duplicates collapse
```

- Set<T> stores unique, unordered elements
- Great for deduplication and fast contains() checks
- union / intersection / difference mirror set theory

45

GENERICS

Writing Reusable, Type-Safe Code

```
class Box<T> {
  T value;
  Box(this.value);
}
var intBox = Box<int>(5);        // T = int
var strBox = Box<String>('hi');  // T = String
```

- <T> is a type placeholder, filled in when the class is used
- List<String>, Map<K,V>, and Future<T> are all generics we've already used
- Generics catch type mistakes at compile time instead of runtime

46

ERROR HANDLING

try, catch & finally

```
try {
  var result = 10 ~/ divisor;
} on IntegerDivisionByZeroException {
  print('Cannot divide by zero');
} catch (e) {
  print('Unexpected error: $e');
} finally {
  print('Done.');
}
```


47

ERROR HANDLING

Custom Exceptions

```
class InsufficientFundsException implements Exception {
  final String message;
  InsufficientFundsException(this.message);
}

throw InsufficientFundsException('Balance too low');
```

- implements Exception marks a class as throwable & catchable
- Custom exceptions carry structured data, not just a String
- on InsufficientFundsException catches only that specific type

48

PREVIEW

A First Look at Future & async/await

```dart
Future<String> fetchUser() async {
  await Future.delayed(Duration(seconds: 2));
  return 'Nora';
}
void main() async {
  print(await fetchUser());  // waits, then: Nora
}
```

- Future<T> represents a value that isn't ready yet
- async marks a function as asynchronous; await pauses for a Future
- Essential for network calls, file I/O, and animations — covered fully later